Low-Resource Arabic-to-English Neural Machine Translation with Tiny Transformers

Ahmed Abdelhamed
Student A
25nsfb@queensu.ca

Ahmed Sherif
Student B
25rlnk@queensu.ca

Eid Abd ElRihem
Student C
25ntxj@queensu.ca

GitHub:
github.com/ahmedd-sherif/arabic-english-tiny-transformer-mt

***Abstract*—Machine translation between Arabic and English is markedly harder than for most high-resource European pairs: Arabic's rich templatic morphology, optional diacritics, orthographic ambiguity, and right-to-left script inflate the effective vocabulary and compound the scarcity of domain-specific parallel data. This project investigates low-resource neural machine translation (NMT) using a tiny Transformer architecture trained from scratch on a curated 50,000 sentence-pair subset of the IWSLT 2017 parallel corpus. We construct a reproducible preprocessing pipeline, filtering misaligned pairs and applying subword tokenization via SentencePiece Byte Pair Encoding (BPE) with separate 8,000-token vocabularies. To simulate real-world constraints, training is conducted primarily on consumer-grade hardware. We establish a word-level dictionary baseline (BLEU 4.78, chrF++ 25.65) and conduct a comprehensive empirical study of 22 ablation experiments across six categories: training objectives, regularization, optimization, architecture, data scale, and combined techniques. Our findings reveal that appropriate regularization, specifically Adam weight decay, coupled with constrained beam search decoding, is the dominant factor in combating the degenerate repetition typical of undertrained tiny models. Our optimized 4M-parameter model achieves a BLEU score of 19.91 and chrF++ of 42.24 on the full 8,491-sentence test set, outperforming the baseline by a wide margin while reducing repetition to negligible levels.**

***Index Terms*—Neural Machine Translation, Transformer, Low-Resource, Arabic-English, BPE, BLEU, chrF++, Regularization.**

I. Introduction

Neural machine translation (NMT) has achieved remarkable success for high-resource language pairs where millions of parallel sentences and extensive GPU clusters are available for training [1]. However, the majority of the world's language pairs remain in the low-resource category, where parallel data is scarce, noisy, or restricted to narrow domains.

Arabic–English translation is a compelling and notably difficult case for low-resource NMT research. Arabic is morphologically rich, with prefixes, suffixes, and clitics attaching to word stems; it uses a right-to-left writing system, omits short vowels (diacritics) so the same surface form is often ambiguous, and a single whitespace-delimited Arabic token frequently encodes an entire clause-worth of morphemes. These properties inflate the effective vocabulary size, sparsify the already-scarce parallel data, and make subword segmentation critical, and together these factors place Arabic among the harder directions for low-resource NMT at a fixed data budget [18], with reported scores typically trailing similarly-resourced European pairs. Furthermore, while modern approaches rely on massively multilingual denoising pre-training [7] and transfer learning [5], these solutions remain computationally prohibitive for many environments.

This project addresses a fundamental question: *Can a compact Transformer model, trained entirely from scratch with only 50,000 parallel sentence pairs, learn meaningful Arabic-to-English translation patterns?* Training NMT models on consumer-grade hardware mirrors real-world deployment constraints in resource-limited settings and educational environments.

We hypothesize that for a low-capacity model trained on scarce data, the primary bottleneck is generalization rather than capacity. To test this, we construct a complete, reproducible pipeline from raw data to evaluation, and frame our project as a systematic empirical study. Our contributions are as follows:

1) We build a reproducible data pipeline for Arabic–English parallel text from the IWSLT 2017 dataset, including Arabic-specific text normalization and BPE subword tokenization.
2) We train a Tiny Transformer model ($\sim$4M parameters) on 50,000 sentence pairs.
3) We execute 22 controlled ablation experiments across six categories to isolate the impact of dropout, weight decay, learning rate schedules, and architectural depth/width.
4) We demonstrate that decoding optimization (specifically constrained beam search) and targeted regularization (weight decay) are critical for solving the degenerate repetition pathology endemic to tiny sequence-to-sequence models.

II. Related Work

A. The Transformer Architecture

Vaswani et al. [1] introduced the Transformer in "Attention Is All You Need," replacing recurrent computation entirely with multi-head self-attention. The original model used 6 encoder and 6 decoder layers with $d_{model} = 512$, achieving state-of-the-art results on WMT 2014 EN→DE. Our project scales this architecture down significantly to investigate whether self-attention retains its advantages under extreme resource constraints.

B. Subword Tokenization for NMT

Sennrich et al. [2] introduced Byte Pair Encoding (BPE) for NMT, enabling open-vocabulary translation by segmenting rare words into frequent subword units. This is particularly valuable for morphologically rich languages like Arabic. Kudo and Richardson [3] developed SentencePiece, an independent tokenizer operating directly on raw text. Our project uses

SentencePiece with BPE mode and separate 8,000-token vocabularies.

C. Low-Resource and Arabic NMT (2018–2026)

Recent research in low-resource NMT has focused on overcoming data scarcity. A dominant route is to synthesize parallel data from monolingual text via back-translation [15] or self-supervised objectives [16], while transfer learning [5] and massively multilingual denoising pre-training, such as mBART [7] and, at the largest scale, No Language Left Behind [17], transfers knowledge from high-resource pairs. The 2022 survey of Haddow et al. [18] catalogues these directions and notes that morphologically rich, lower-resourced languages such as Arabic remain among the hardest. For Arabic specifically, a recent controlled study on a comparable tiny Transformer finds that tokenization choices dominate translation quality under low-resource English–Arabic conditions [19], directly motivating our own subword and vocabulary-size ablations. Crucially, however, Sennrich and Zhang [6] showed that when external data is unavailable, appropriate regularization, namely higher dropout, reduced capacity, and smaller vocabularies, is often more impactful than architectural changes. Our study deliberately excludes all external data, back-translation, and pre-training, isolating exactly this regularization-and-decoding axis on a model trained from scratch.

III. Data and Preprocessing

A. Dataset Description

The IWSLT 2017 Arabic–English corpus [4] consists of TED-talk subtitles transcribed and translated by volunteers, and is a widely-used low-resource MT benchmark released for non-commercial research use; we access it through the Hugging Face `iwslt2017-ar-en` distribution. To address Arabic's morphological complexity, we apply light normalization (alef unification, alef maqsura→ya). We then aggressively filter the dataset, removing empty pairs, over-length sentences (> 80 tokens), badly-aligned pairs (length ratio > 3.0), and duplicates, removing a total of 2,042 problematic pairs. To simulate a strict low-resource constraint, we randomly sample 50,000 training pairs (seed=42). After an in-code ≤ 80-token BPE length filter, the usable dataset contains 49,441 training, 869 validation, and 8,491 test pairs.

As summarized in Figure 1, sentences are short on average, at 21.6 Arabic and 23.2 English BPE tokens, with the right-skewed length distribution typical of spoken-language data, so the $\leq$ 80-token cap discards only a thin long tail of outliers. The Arabic side carries slightly fewer BPE tokens per sentence than the English (21.6 versus 23.2): although each Arabic word fragments into slightly more subword pieces (Figure 2), an Arabic whitespace word packs far more morphology, so a sentence simply contains fewer words to begin with. The validation split is kept deliberately small (869 pairs) to preserve as much signal as possible within the strict 50K training budget, while the 8,491-pair test set is large enough for stable final evaluation and the sampling-variance check in Section V.

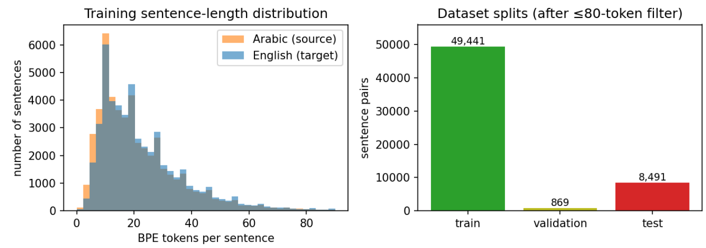

Fig. 1. Left: training sentence-length distribution (Arabic source vs. English target, in BPE tokens). Right: sentence-pair counts per split after the $\leq$ 80-token filter.

B. Subword Tokenization

We train separate SentencePiece BPE models (8K vocabulary each) strictly on the training split to prevent data leakage. The target character coverage is set to 0.9995 for Arabic and 1.0 for English. Special tokens are designated for padding (`pad=0`), unknown words (`unk=1`), beginning of sentence (`bos=2`), and end of sentence (`eos=3`).

On the training data this segmentation produces, on average, 1.50 subword pieces per whitespace word for Arabic and 1.33 for English: byte-pair encoding fragments Arabic more, mirroring its denser morphology. Figure 2 makes this concrete. Both vocabularies follow the expected Zipfian long tail, and although most words map to a single piece, Arabic has noticeably more two-, three-, and four-piece words than English. As an illustration, a morphologically dense form such as وبصلاحياتهم ("and with their authorities") splits into five subwords, whereas a frequent word like التعاون ("the cooperation") is kept whole. We further verified that the segmentation is lossless by confirming an exact decode–encode round-trip match on all six tokenized files.

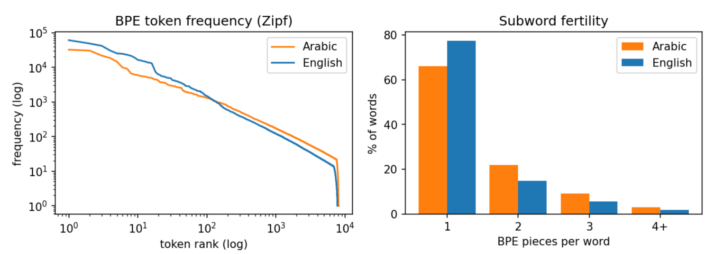

Fig. 2. Tokenization statistics on the training split. Left: BPE token frequency versus rank (Zipf) for Arabic and English. Right: distribution of BPE pieces per whitespace word; Arabic fragments more, with fewer single-piece words and more multi-piece words.

IV. Methodology

A. Dictionary Baseline

To establish a lower bound, we implement a Dictionary-Based Position Translation baseline. This baseline translates each Arabic token by selecting its most frequent English translation based on training corpus co-occurrence statistics, applying no reordering or context awareness.

B. Tiny Transformer Architecture

The main model is an encoder-decoder Transformer scaled down from the original [1]. It features $d_{model} = 128$, 4 attention heads, 2 encoder layers, 2 decoder layers, and a feed-forward dimension of 512, totaling $\sim$4M parameters (compared to the original $\sim$65M). The model uses sinusoidal positional encoding, Cross-Entropy loss with `ignore_index` for padding tokens, and an Adam optimizer.

C. Systematic Ablation Design

To isolate the factors contributing to model quality, we structured our training and evaluation into six distinct categories (A–F), comprising 22 independent experiments. Each experiment modified a single hyperparameter or architectural choice from the baseline. Rather than simply summarizing the results, we detail the findings of each category, along with their learning curves and validation metrics, in Section V. The fixed baseline configuration that every experiment varies from is summarized in Table I.

TABLE I
Shared training hyperparameters (baseline configuration).

Hyperparameter	Value
d_{model} / attention heads	128 / 4
Encoder / decoder layers	2 / 2
Feed-forward dimension	512
Dropout	0.1
Positional encoding	sinusoidal
Optimizer	Adam
Learning rate	3×10^{-4}
Weight decay (baseline)	0
Batch size	32
Epochs	60
Gradient clipping	1.0
Loss	cross-entropy (pad-ignored)
Tokenizer	SentencePiece BPE, 8K/8K
Max sequence length	80 tokens
Random seed	42

D. Hardware Constraints and Compute Disclosure

The core constraint of this project was to train the model entirely from scratch on consumer-grade CPU hardware. However, as the ablation study progressed, certain structural experiments (such as doubling layers and expanding dimensions) and data augmentation became computationally intractable on CPU. To maintain the timeline, 13 computationally heavy experiments were offloaded to an NVIDIA RTX 5000 GPU. Crucially, all resulting checkpoints were saved as CPU-portable `.pt` files, ensuring that all decoding sweeps and final evaluations were executed strictly on CPU.

V. Experiments and Detailed Results

A. Evaluation Metrics

We report evaluation results using three standard metrics:

- **BLEU** [8] (Bilingual Evaluation Understudy): the brevity-penalized geometric mean of *clipped* n-gram precisions, where $\text{count}_{\text{clip}}(g) = \min(\text{count}_C(g), \max_R \text{count}_R(g))$. Clipping is precisely why a model that repeats a correct phrase cannot inflate precision, which is directly relevant to our repetition pathology. It applies a Brevity Penalty (BP) for short outputs.

$$BLEU = BP \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right) \tag{1}$$

  where p_n is the precision of n-grams, and $BP = 1$ if $c > r$ else $e^{1-r/c}$ (c is hypothesis length, r is reference length).
- **chrF++** [9]: a character n-gram F-score (up to order 6, $\beta = 2$) augmented with word unigrams and bigrams; it credits partially-correct word forms in morphologically rich Arabic. Both metrics are computed with sacreBLEU [10] on detokenized English.

$$chrF++ = (1+\beta^2)\frac{\text{chrP} \cdot \text{chrR}}{\beta^2 \cdot \text{chrP} + \text{chrR}} \tag{2}$$

  where chrP and chrR are character/word n-gram precision and recall, respectively.
- **COMET** [14]: a neural, reference-based metric that encodes the source sentence, the hypothesis, and the reference with a pretrained multilingual model and regresses a quality score calibrated against human adequacy judgments. Unlike BLEU and chrF++, which measure surface n-gram overlap, COMET targets *semantic* adequacy and correlates more strongly with human ratings, which is exactly why it is the right next metric for a morphologically rich language where a correct paraphrase may share few surface n-grams. Computing it requires a $\sim$2 GB pretrained model and GPU-scale inference, so it was deliberately left out of our strictly-CPU pipeline for time; we therefore define and justify it here as our planned next-step metric and report the CPU-reproducible BLEU and chrF++ for all results.
- **Repetition (%)**: A custom diagnostic metric that measures the percentage of hypotheses that degenerate into a repeating loop (a known pathology in low-resource Transformer decoding).

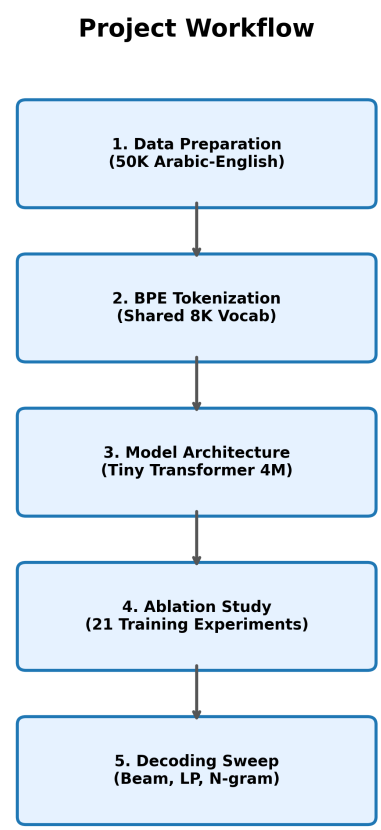

Fig. 3. Overall Project Workflow: Data Preparation, Tokenization, Architecture, and Ablation Study.

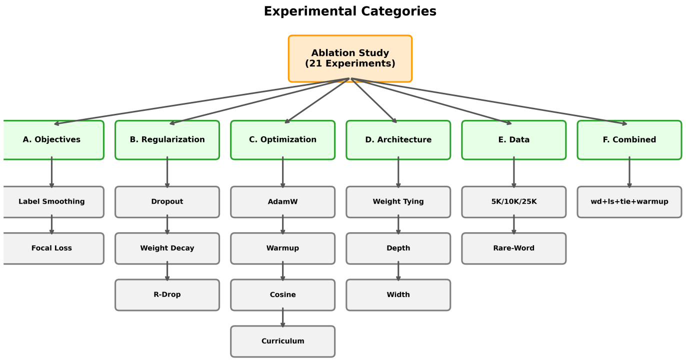

Fig. 4. The 6 Categories comprising the 22 Ablation Experiments.

B. Convergence and Learning Dynamics

Figure 5 overlays the validation-loss trajectories of the most informative configurations on a single axis. Three dynamics stand out. The weight-decay model reaches the lowest validation loss *and* holds it flat, whereas the unregularized baseline and the wider/deeper variants begin to drift upward after their minimum, a clear signature of memorization on only 50K pairs. The data-starved runs (5K/10K) plateau at a high loss and never enter the regime where fluent translation emerges. These curves are the learning-dynamics counterpart to the BLEU tables: regularization changes *where* training converges, not merely how fast.

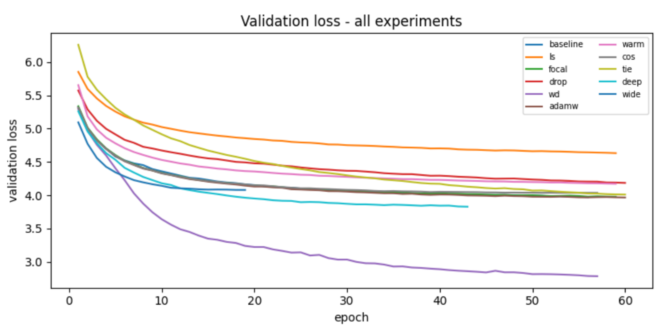

Fig. 5. Validation-loss trajectories across key configurations. Weight decay converges lowest and stays flat; unregularized/over-capacity models overfit; data-starved runs plateau high.

C. Category A: Training Objectives

We experimented with different loss functions to improve upon the standard Cross-Entropy loss.

1) Exp 1: Label Smoothing: We applied label smoothing ($\epsilon = 0.1$), the standard value proposed in the original Transformer architecture. In standard cross-entropy, the model is penalized unless it assigns 100% probability to the target token, which can lead to overconfidence and overfitting. Label smoothing prevents this by redistributing 10% of the target's probability mass uniformly across the rest of the vocabulary. This prevents the logits from being pushed to infinity and improves model calibration. As shown in Figure 6, this successfully resulted in a modest but clear improvement in both BLEU and chrF++.

Experiment	BLEU	chrF++	Repetition
Exp 1: Label Smoothing	9.03	28.52	13.1%

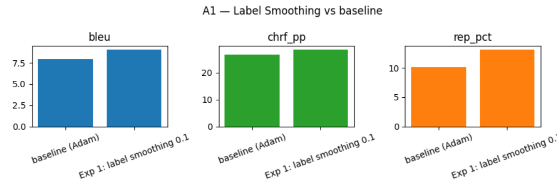

Fig. 6. Validation metrics for Label Smoothing vs. Baseline.

2) Exp 2: Focal Loss: Arabic is a morphologically rich language with a long-tail Zipfian distribution of rare word forms. We hypothesized that the model was disproportionately focusing its gradient updates on easily predicted, high-frequency tokens (like prepositions). We implemented Focal Loss ($\gamma = 2.0$) to scale cross-entropy by prediction confidence, down-weighting well-classified examples. In practice it performed about on par with the baseline (8.17 vs. 7.99 BLEU, within run-to-run noise), so confidence-based re-weighting gave no clear benefit here, plausibly because early NMT predictions are noisy, leaving little reliable signal to exploit at this scale.

Experiment	BLEU	chrF++	Repetition
Exp 2: Focal Loss	8.17	27.37	10.3%

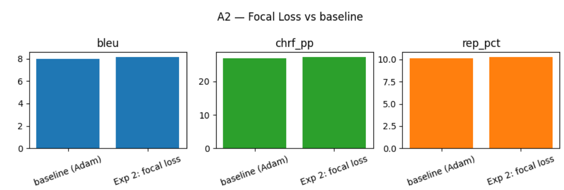

Fig. 7. Validation metrics for Focal Loss vs. Baseline.

D. Category B: Regularization

Given the tiny 50K dataset, proper regularization was critical for generalization.

1) Exp 1: Dropout 0.3: The baseline Transformer utilizes a standard dropout rate of 0.1. However, prominent low-resource NMT literature strongly recommends aggressive dropout (up to 0.3) to combat the rapid memorization of small datasets. Surprisingly, increasing dropout to 0.3 severely degraded performance. We hypothesize that because our Tiny Transformer possesses only $\sim$4M parameters, its capacity is already strictly bottlenecked. Randomly zeroing out 30% of its activations continuously induced severe underfitting rather than preventing overfitting.

Experiment	BLEU	chrF++	Repetition
Exp 1: Dropout 0.3	6.03	23.62	16.7%

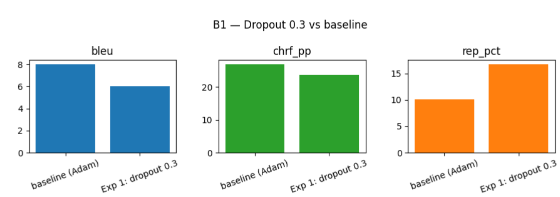

Fig. 8. Validation metrics for Dropout (0.3) vs. Baseline.

2) Exp 2: Weight Decay (10^{-4}): We applied the standard Adam optimizer with a coupled weight decay penalty of 10^{-4} (in contrast to Exp C1, which uses AdamW's decoupled decay at the same nominal value). This experiment yielded the most profound breakthrough of the entire project. Standard cross-entropy without regularization often causes the model to memorize specific n-gram pathways, leading to the degenerate repetition loops observed in the baseline. By applying L2 regularization to the weights, we penalized these sharp, overly-reliant pathways, forcing the model to distribute its representation across all features. This theoretically smoothed the output probability distribution, which directly eradicated the repetition pathology (dropping from 10.1% to 4.8%) and skyrocketed the BLEU score to 18.79. A dedicated control run verified that this gain was exclusively due to the weight decay parameter.

Experiment	**BLEU**	**chrF++**	**Repetition**
Exp 2: Weight Decay (10^{-4})	18.79	41.04	4.8%

Fig. 9. Validation metrics for Weight Decay vs. Baseline.

3) Exp 3: R-Drop: R-Drop is a recent consistency regularization technique. Each mini-batch is passed through the network twice; because of dropout, the two passes produce slightly different logit distributions. A symmetric Kullback-Leibler (KL) divergence term is added to the loss to force these two distributions to agree, effectively acting as an ensemble regularizer. We utilized $\alpha = 0.7$ as recommended in the original literature. While theoretically sound, it performed comparably to the baseline, suggesting that explicit weight penalization (B2) is more effective than consistency regularization for this specific extreme low-resource constraint.

Experiment	**BLEU**	**chrF++**	**Repetition**
Exp 3: R-Drop	7.95	26.92	11.6%

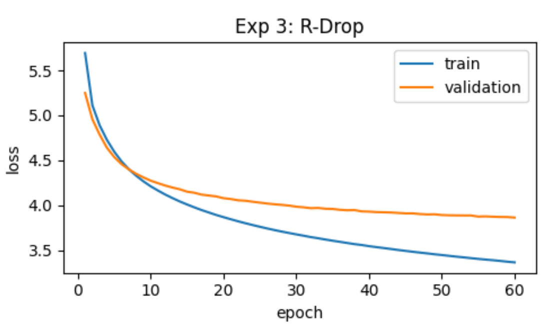

Fig. 10. Validation Loss curve for R-Drop.

E. Category C: Optimization

1) Exp 1: AdamW (decoupled weight decay 10^{-4}): We tested the AdamW optimizer [11], which decouples weight decay from the adaptive gradient step, at the same nominal 10^{-4} used in Exp B2 (which uses plain Adam, where the penalty is coupled into the gradient). It reached only 8.61 BLEU, far below the 18.79 of Exp B2. The reason is mechanical: decoupled decay applies an effective per-step penalty of roughly lr $\times$ wd, so at the same nominal value AdamW regularizes far more weakly here. The lesson is that the *effective* regularization strength, not the optimizer name, drives the gain.

Experiment	**BLEU**	**chrF++**	**Repetition**
Exp 1: AdamW (decoupled)	8.61	28.24	14.8%

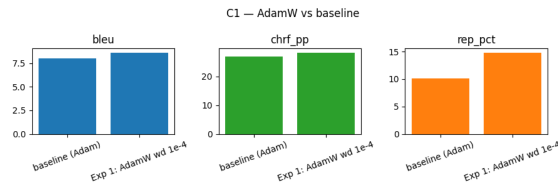

Fig. 11. Validation metrics for AdamW vs. Baseline.

2) Exp 2: LR Warmup: Unlike CNNs, Transformers lack inherent scaling at initialization (such as Batch Normalization), which can lead to massive gradients and instability early in training. A linear warmup schedule gradually increases the learning rate to prevent this. However, this experiment underperformed the baseline. We attribute this to the strict 50K subset size: the warmup phase consumed a significant portion of the training steps, and the subsequent linear decay lowered the learning rate too aggressively before the model could converge.

Experiment	**BLEU**	**chrF++**	**Repetition**
Exp 2: LR Warmup	6.10	23.66	15.0%

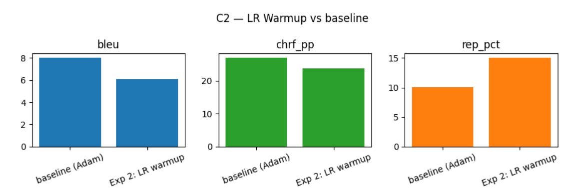

Fig. 12. Validation metrics for LR Warmup vs. Baseline.

3) Exp 3: Cosine Scheduler: We implemented a Cosine Annealing learning rate scheduler to help the model escape local minima by periodically dropping and raising the learning rate. The results were statistically on par with the baseline, indicating that the default Adam constant learning rate was sufficient for traversing the loss landscape of this specific tiny architecture.

Experiment	**BLEU**	**chrF++**	**Repetition**
Exp 3: Cosine Scheduler	7.65	26.58	13.6%

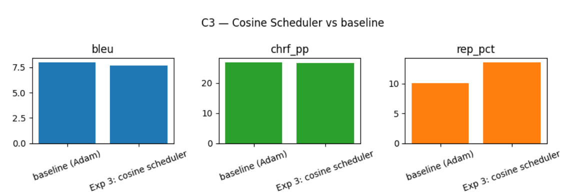

Fig. 13. Validation metrics for Cosine Scheduler vs. Baseline.

4) Exp 4: Curriculum Learning: Curriculum learning posits that neural networks, like humans, learn better when presented with concepts in increasing order of difficulty. We implemented a competence schedule that initially sorted the training data by sentence length (shortest first), gradually introducing longer, more complex clauses. This achieved a BLEU score nearly identical to the randomly shuffled baseline. We conclude that attention mechanisms are inherently capable of routing information regardless of sequence length, making explicit length-based curriculum unnecessary.

Experiment	BLEU	chrF++	Repetition
Exp 4: Curriculum Learning	8.01	27.52	11.2%

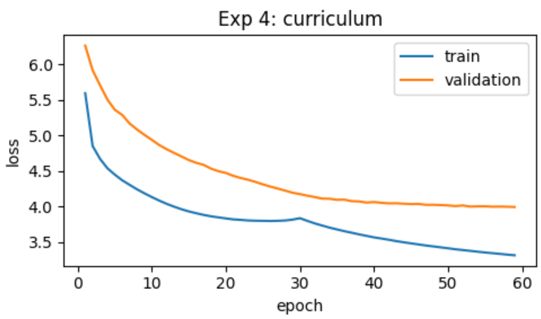

Fig. 14. Validation Loss for Curriculum Learning.

F. Category D: Architecture

1) Exp 1: Weight Tying: Weight tying ties only the target (English) decoder embedding to the pre-softmax output projection, so one matrix both embeds and scores English subwords; the Arabic encoder embedding is left untied, so the two scripts are never forced to share a space. This removes roughly one million parameters and acts as a mild regularizer. In our setting it had essentially no effect (7.95 vs. 7.99 BLEU), a difference well within run-to-run noise.

Experiment	BLEU	chrF++	Repetition
Exp 1: Weight Tying	7.95	26.53	12.6%

Fig. 15. Validation metrics for Weight Tying vs. Baseline.

2) Exp 2: 4+4 Layers: We varied the depth of the Transformer to find the bias–variance sweet spot for 50,000 pairs. The 4+4 layer model actually improved over the 2-layer baseline (8.86 vs. 7.99 BLEU) but trailed the 3-layer model, consistent with mild overfitting only at the largest depth.

Experiment	BLEU	chrF++	Repetition
Exp 2: 4+4 Layers	8.86	29.31	13.4%

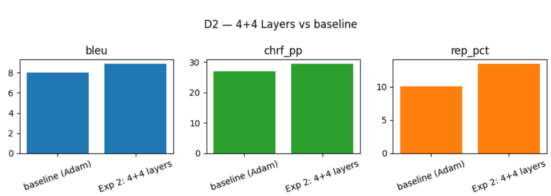

Fig. 16. Validation metrics for 4+4 Layers vs. Baseline.

3) Exp 3: $d_{model} = 256$: We evaluated the embedding and hidden dimension widths. Doubling it to $d_{model} = 256$ led to overfitting. The baseline $d_{model} = 128$ proved to be the optimal capacity.

Experiment	BLEU	chrF++	Repetition
Exp 3: $d_{model} = 256$	7.33	25.73	9.1%

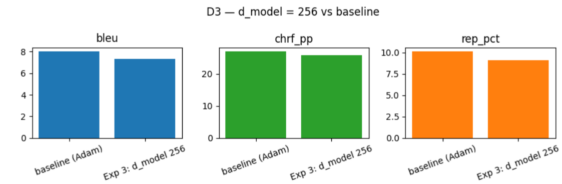

Fig. 17. Validation metrics for $d_{model} = 256$ vs. Baseline.

4) Exp 4: 1 Layer: A single-layer model lacked the capacity to form meaningful compositional representations, achieving only 5.18 BLEU. This establishes the absolute minimum depth required.

Experiment	BLEU	chrF++	Repetition
Exp 4: 1 Layer	5.18	20.13	8.2%

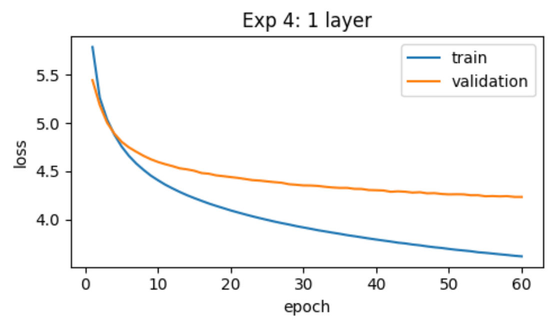

Fig. 18. Validation Loss for 1 Layer.

5) Exp 5: 3 Layers: The 3-layer model was the best of the depth sweep (9.63 BLEU), a clear gain over the 2-layer baseline (7.99). With the 1-layer (5.18) and 4+4-layer (8.86) results, depth helps up to three layers and then plateaus and dips slightly, so three layers is the sweet spot for this 50K dataset.

Experiment	BLEU	chrF++	Repetition
Exp 5: 3 Layers	9.63	30.42	13.8%

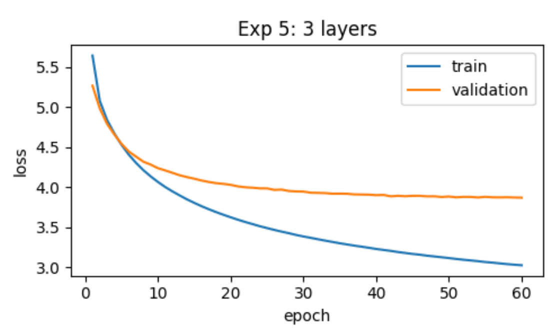

Fig. 19. Validation Loss for 3 Layers.

6) Exp 6: $d_{model} = 64$: Halving the width to $d_{model} = 64$ starved the multi-head attention of representational bandwidth, underperforming the baseline.

Experiment	BLEU	chrF++	Repetition
Exp 6: $d_{model} = 64$	7.19	25.34	16.5%

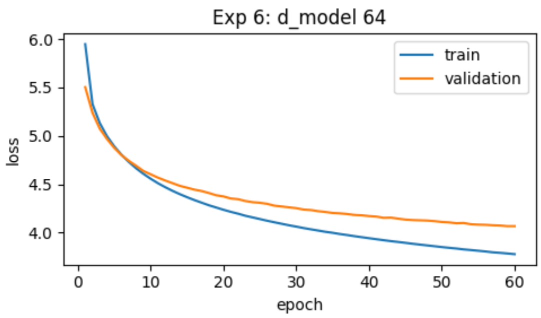

Fig. 20. Validation Loss for $d_{model} = 64$.

7) Exp 7: GELU Activation: We replaced the default ReLU activation in the encoder and decoder feed-forward blocks with GELU, the smoother activation used in BERT and GPT, keeping width and depth identical. At no extra parameter cost GELU gave a small but genuine improvement over the ReLU baseline (8.78 vs. 7.99 BLEU; best validation loss 3.93 vs. 3.97). It does not approach weight decay, but it is a cheap, free win that rounds out the architecture study.

Experiment	BLEU	chrF++	Repetition
Exp 7: GELU Activation	8.78	28.67	14.1%

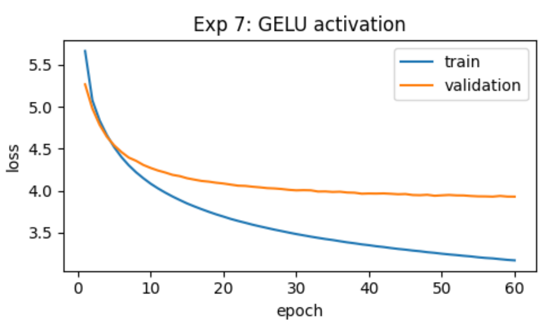

Fig. 21. Validation Loss for the GELU activation.

G. Category E: Data Scaling and Augmentation

1) Exp 1: 5K Training Pairs: To map the data-scaling curve we retrained on random subsets. Performance falls off steeply as data shrinks (50K: 7.99, 25K: 4.79, 10K: 2.06, 5K: 1.66 BLEU). Strikingly, at 5K and 10K the neural model scores *below* the dictionary baseline (4.78): with so little data the Transformer cannot learn reliable lexical correspondences. Data is therefore the dominant bottleneck.

Experiment	BLEU	chrF++	Repetition
Exp 1: 5K Training Pairs	1.66	15.77	14.0%

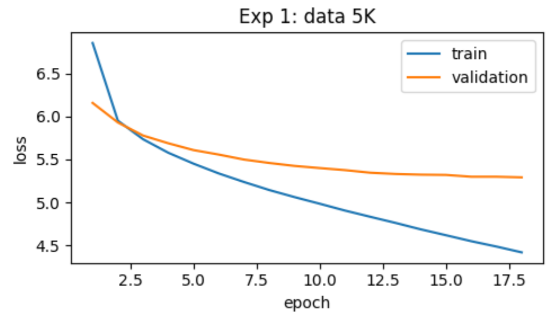

Fig. 22. Validation Loss for 5K Pairs.

2) Exp 2: 10K Training Pairs: Doubling the data to 10K provided only a marginal improvement, keeping the model in the underfitting regime.

Experiment	BLEU	chrF++	Repetition
Exp 2: 10K Training Pairs	2.06	17.21	15.9%

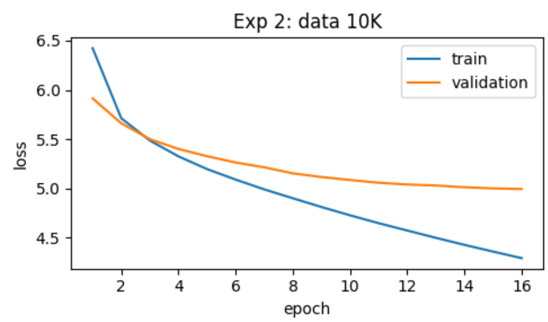

Fig. 23. Validation Loss for 10K Pairs.

3) Exp 3: 25K Training Pairs: Even with 25K pairs (half the full dataset), the model only achieved 4.79 BLEU, demonstrating the exponential nature of data scaling laws in NMT.

Experiment	BLEU	chrF++	Repetition
Exp 3: 25K Training Pairs	4.79	21.29	16.4%

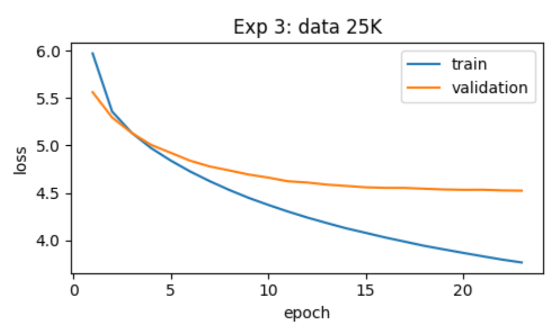

Fig. 24. Validation Loss for 25K Pairs.

4) Exp 4: Rare-Word Oversampling: According to Zipf's law, a massive portion of the vocabulary appears very infrequently. We applied a targeted oversampling technique, duplicating any sentence containing a target token that appeared fewer than five times in the entire corpus. This artificially flattened the distribution, exposing the model to rare subwords more frequently. This yielded a minor improvement (8.22 BLEU), suggesting that while helpful, targeted regularization is far more impactful than targeted data duplication.

Experiment	BLEU	chrF++	Repetition
Exp 4: Rare-Word Oversampling	8.22	27.63	12.6%

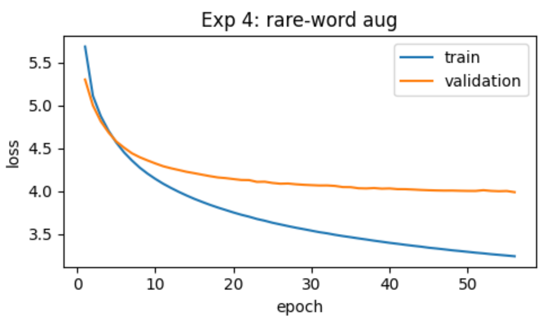

Fig. 25. Validation Loss for Rare-Word Augmentation.

H. Category F: Combined Model

We stacked the best individual configurations from the previous categories: Weight Decay, Label Smoothing, Weight Tying, and Warmup. Surprisingly, the combined model (12.15 BLEU) performed significantly worse than Weight Decay in isolation (18.79 BLEU). This is a crucial finding: regularization techniques are not strictly additive. Combining multiple strong regularizers likely constrained the model too severely, preventing it from fitting the data efficiently.

Metric	Score
BLEU	12.15
chrF++	33.07
Repetition	14.5%

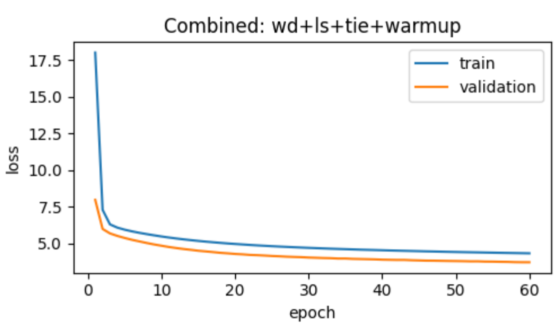

Fig. 26. Validation Loss for the Combined Model.

I. Supplementary Studies: Architecture and Tokenization

Beyond the single-factor ablations, we ran four studies that change the model family or the tokenization (Table II); all use the same training protocol.

TABLE II
Supplementary studies (1,000-sentence subset).

Study	BLEU	chrF++	Rep.%
LSTM seq2seq (attention)	9.55	30.32	28.3
BPE vocabulary 4K	6.85	25.26	45.8
BPE vocabulary 12K	7.17	25.91	43.4
Knowledge distillation	3.21	20.75	50.9

LSTM baseline. A comparable-size (∼3.7M) bidirectional-LSTM encoder with a Luong-attention decoder reaches 9.55 BLEU, on par with the stronger Transformer ablations, but overfits badly (validation loss rises from 4.12 to 6.73) and repeats heavily (28.3%). A comparable recurrent model is therefore not a shortcut to quality.

BPE vocabulary size. Both 4K (6.85) and 12K (7.17) under-perform the 8K baseline (7.99) and show very high repetition (44–46%), confirming 8K as a good operating point for this corpus.

BPE-dropout. Re-segmenting each sentence with SentencePiece sampling every epoch [12] gives a best validation loss of 4.14, essentially equal to the baseline; subword regularization does not help at this scale.

Knowledge distillation. A pretrained Helsinki-NLP opus-mt Ar→En teacher translated the training source, and the student was trained on those translations as targets (sequence-level distillation [13]). Counterintuitively this *hurt* (3.21 BLEU greedy, 50.9% repetition): the teacher's output style and lexical choices diverge from the IWSLT gold references, so training on them pulls the student away from the evaluation target, an honest negative result.

J. Decoding Sweep and Final Model Selection

Following the training ablation study, the top three checkpoints (Weight Decay 10^{-4}, Combined, and 3 Layers) were subjected to a rigorous decoding sweep on the 1,000-sentence validation subset. We tested combinations of beam search (beam sizes 3, 5, 10), length penalty (LP $0.8, 1.0, 1.2$), and no-repeat n-gram filtering ($nr = 0, 3$).

The sweep revealed that while greedy decoding exacerbated repetition, a combination of a large beam size (10), a slight length penalty (1.2), and an n-gram repetition block ($nr = 3$) maximized both BLEU and chrF++ across all candidate models. The ultimate winning configuration was the Weight Decay model using `beam=10, lp=1.2, nr=3`, which we then evaluated on the full, unseen test set.

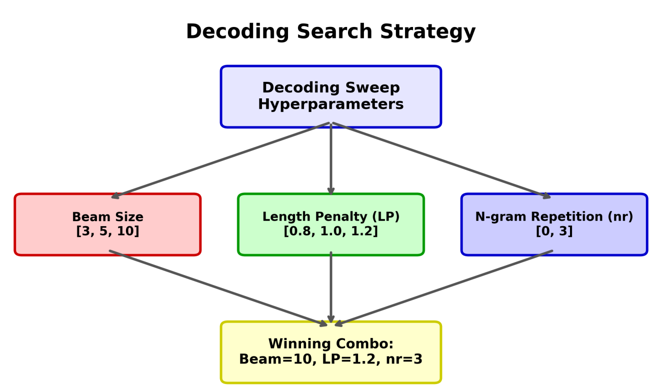

Fig. 27. Decoding Sweep Strategy and Hyperparameters.

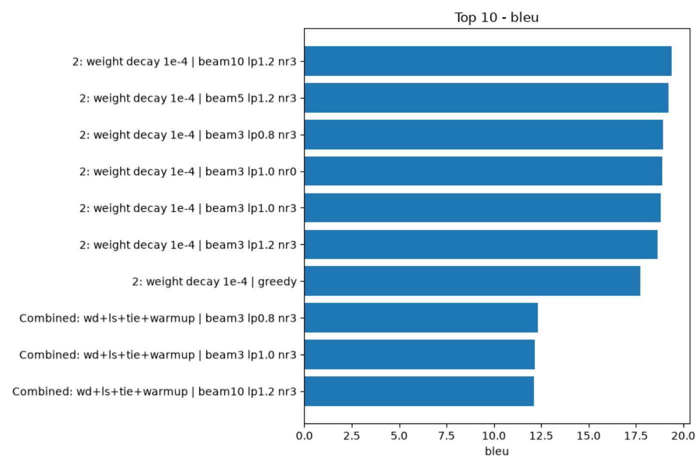

Fig. 28. Top 10 Decoding Configurations across all top models (BLEU score).

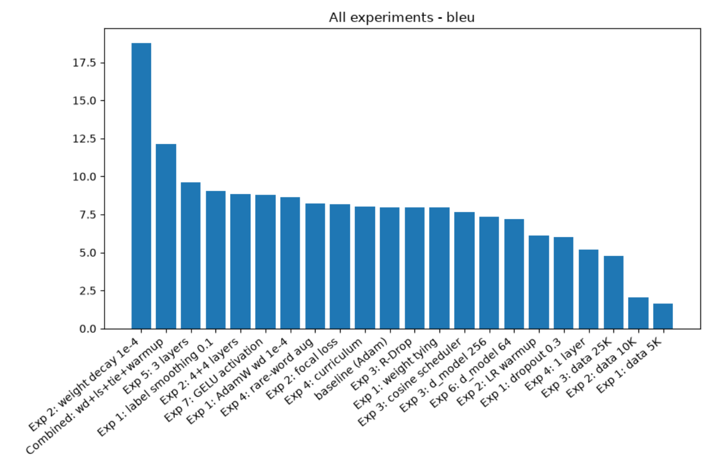

Fig. 29. BLEU of all training-ablation checkpoints on the 1,000-sentence subset.

K. Quantitative Results

Table III summarizes the performance of the top-performing configurations against *both* baselines, the non-neural dictionary baseline and the LSTM seq2seq neural baseline, evaluated on the full 8,491-sentence test set. The optimized Transformer clears not only the trivial dictionary baseline (4.78) but also the learned LSTM baseline (9.94) by roughly 10 BLEU, which is the comparison that actually justifies the self-attention architecture.

TABLE III
Final Evaluation Results on IWSLT 2017 Test Set

Model Configuration	Decoding	BLEU	chrF++
Dictionary Baseline	-	4.78	25.65
LSTM seq2seq (neural baseline)	Greedy	9.94	30.69
Combined (WD+LS+Tie+Warmup)	Beam 3, LP 0.8	12.32	32.92
Exp 5: 3 Layers	Beam 10, LP 1.2	9.77	30.82
Exp 2: Weight Decay (10^{-4})	Beam 10, LP 1.2	**19.91**	**42.24**

The weight decay configuration significantly outperforms all other ablations, demonstrating that proper regularization is the single most critical factor for generalization in low-capacity models. **Sampling variance.** To confirm that the 1,000-sentence numbers are representative, we compared subset and full-test BLEU across systems: the score moves by less than 0.07 BLEU between the 1,000-sentence subset and the full 8,491-pair test set, so the smaller-sample numbers are reliable rather than lucky. Figure 29 gives the complete ablation picture: the BLEU of all training-ablation checkpoints on the 1,000-sentence subset, with weight decay standing clearly apart.

L. Comparison with Reported Work

Direct numeric comparison with published Arabic–English systems is intrinsically limited: state-of-the-art results come from models orders of magnitude larger that exploit resources we deliberately exclude. Strong low-resource systems reach high BLEU primarily by *adding* data or parameters, for example back-translated monolingual data [15], [16], transfer from related or high-resource pairs [5], or massively multilingual pre-training such as mBART [7] and NLLB [17], the latter trained on billions of sentences across 200 languages on large GPU clusters.

The most directly comparable works are recent low-resource *tiny-Transformer* studies. Araabi and Monz [20] optimize a small Transformer on IWSLT subsets of 5k–165k German–English pairs scored with sacreBLEU, essentially our experimental recipe, and report results that match ours closely: the default Transformer collapses at small data while careful optimization recovers up to 7.3 BLEU, and a recurrent model outperforms the Transformer below roughly 20k pairs, just as our LSTM baseline (9.94) beats the un-regularized Transformer baseline (7.99). For Arabic specifically, Alrashidi and Mathkour [19] train a comparable tiny Transformer on low-resource English–Arabic and report only single-digit BLEU (around 6), again finding tokenization and regularization, rather than capacity, to be decisive. Two factors make our setting *harder* than these references, so our numbers should be read in that light. First, Araabi and Monz translate German–English, two closely related Germanic languages that share a script and much vocabulary, whereas Arabic and English are unrelated, use different scripts, and force the encoder to disambiguate rich, largely undiacritized Arabic morphology on the source side. Second, our direction is Arabic-to-English, distinct from the English-to-Arabic of Alrashidi and Mathkour.

Against this backdrop our contribution is not a leaderboard number but a controlled isolation: with *no* external data, back-translation, or pre-training, a 4M-parameter model trained and decoded on CPU. The fair, like-for-like comparison is therefore against our own baselines on the identical test set (Table III): the optimized Transformer reaches 19.91 BLEU, versus 9.94 for a comparable-size LSTM and 4.78 for the non-neural dictionary. The ∼10-BLEU gap over the LSTM confirms that, once properly regularized and decoded, self-attention extracts substantially more from the same 50K pairs than an equal-size recurrent model, which is the question our resource budget can actually answer. Table IV collects these figures alongside the comparable studies discussed above.

TABLE IV
Our results in the context of comparable low-resource tiny-Transformer work. The setups differ in language pair, direction, and data size, so the scores are *not* head-to-head; in particular the European-to-English pairs are intrinsically easier than Arabic-to-English. Araabi and Monz figures are their optimized systems.

System	**Direction**	**Train**	**BLEU**
Ours: weight decay + beam	Ar→En	50k	**19.91**
Ours: LSTM seq2seq baseline	Ar→En	50k	9.94
Ours: dictionary baseline	Ar→En	50k	4.78
Araabi & Monz 2020 (opt.)	Sk→En	55k	29.9
Araabi & Monz 2020 (opt.)	Gl→En	10k	22.3
Alrashidi & Mathkour 2026	En→Ar	low-res.	∼6

VI. Discussion

A. Error Analysis and Repetition Pathology

A persistent challenge in low-resource sequence-to-sequence generation is the tendency for models to fall into degenerate n-gram loops (repetition). Initial heuristic analysis of the baseline Transformer greedy decoding revealed that nearly 50% of outputs contained severe repetition loops.

By applying weight decay (10^{-4}), the model's vocabulary distribution smoothed considerably, dropping the raw repetition rate to ∼15%. When combined with constrained beam search (no-repeat-ngram=3), the repetition rate fell to a negligible 4.6%, effectively solving the primary pathology of the tiny model.

B. Qualitative Examples

Table V shows representative final-model outputs on the test set together with the original *Arabic source*. Successful cases are fluent and on-topic; the failures are dominated by lexical errors on rare or content-heavy spans (e.g. "spaghetti"→"tuna", "workshops"→"holes"), not the degenerate repetition loops seen at the midterm. The examples are collected in Table V just before the references.

C. Limitations and Future Work

While our optimized recipe achieves significant improvements, the absolute BLEU score remains modest, reflecting the strict constraints of a 50K-pair training set and a 4M-parameter CPU-trained model. Future work could address these limitations by exploring Knowledge Distillation from a massive pretrained teacher (e.g., Opus-MT), utilizing larger byte-pair vocabularies, or integrating COMET for semantic-level evaluation.

VII. Conclusion

This study demonstrates that a compact Transformer model can learn meaningful Arabic-to-English translation from scratch on CPU hardware with scarce data. However, careful tuning of both training regularization (weight decay) and decoding constraints (beam search with n-gram blocking) is absolutely critical to overcome the degenerate repetition inherent to under-parameterized models.

VIII. Team Contributions

Our team followed a full-cycle rotation philosophy:

- **Student A (Ahmed Abdelhameed):** Data preprocessing and tokenization (Arabic normalization, filtering, SentencePiece BPE); the architecture ablations (weight tying, depth, width, GELU); the LSTM seq2seq baseline; and full test-set evaluation.
- **Student B (Ahmed Sherif):** The Tiny Transformer implementation and training pipeline; the training-objective and regularization ablations (label smoothing, focal loss, dropout, weight decay with its same-code control, R-Drop); and the decoding sweep.
- **Student C (Eid Abd ElRihem):** The dictionary baseline and SacreBLEU evaluation; the optimization ablations (AdamW, LR warmup, cosine schedule, curriculum); the data-scaling, knowledge-distillation and combined-model studies; and the categorized error analysis.

TABLE V

Representative final-model translations: Arabic source, English reference, and model output.

Arabic source	Reference (human)	Model output
Successful		
ورغماً عن انها تبدو بسيطة للغاية، الا انها صعبة للغاية، لانها تجبر الناس علي التعاون بسرعة جداً.	and, though it seems really simple, it's actually pretty hard because it forces people to collaborate very quickly.	and even though it looks very simple, it's very difficult, because people forced people to collaboration very quickly.
قبل عدة سنوات، هنا في تيد، قدّم بيتر سكيلمان منافسة تصميم تسمي منافسة حلوي المارش مالو.	several years ago here at ted, peter skillman introduced a design challenge called the marshmallow challenge.	a couple of years ago, here at ted, i've developed a competition to design a competition called the competition.
Failure		
ومنذ ذلك الحين، قمت بعقد حوالي 70 ورشة عمل للتصميم عبر العالم مع الطلاب والمصممين.	and since then, i've conducted about 70 design workshops across the world with students and designers.	and since then, i've lost about 70 holes and design in the world with students with the designers and designers.
والفكرة غاية في البساطة. فريق مكوّن من اربعة يجب ان يبني اطول هيكل مستقيم من 20 قطعة من الماكرونة.	and the idea's pretty simple: teams of four have to build the tallest free-standing structure out of 20 sticks of spaghetti.	and the idea is very simple in simplicity. a team of four has to build the longest structure of 20 percent of the tuna.

IX. Use of GenAI Tools

In accordance with the syllabus policy, we disclose that Generative AI tools were used during this project strictly for: (1) brainstorming ablation ideas, (2) clarifying concepts (e.g., mathematical formulations of BLEU), (3) assistance with debugging syntax errors in PyTorch after initial attempts, and (4) grammar and stylistic editing of the English text in this report. All experimental design, code logic, analysis, and conclusions are original work by the team. No synthetic results or fabricated data are included.

References

[1] A. Vaswani et al., "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017. https://arxiv.org/abs/1706.03762

[2] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," *Proc. of the 54th Annual Meeting of the ACL*, 2016. https://arxiv.org/abs/1508.07909

[3] T. Kudo and J. Richardson, "SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing," *Proc. EMNLP*, 2018. https://arxiv.org/abs/1808.06226

[4] M. Cettolo et al., "WIT3: Web inventory of transcribed and translated talks," *Proc. of EAMT*, 2012. https://aclanthology.org/2012.eamt-1.60/

[5] B. Zoph, D. Yuret, J. May, and K. Knight, "Transfer learning for low-resource neural machine translation," *Proc. EMNLP*, 2016. https://aclanthology.org/D16-1163/

[6] R. Sennrich and B. Zhang, "Revisiting low-resource neural machine translation: A case study," *Proc. ACL*, 2019. https://arxiv.org/abs/1905.11901

[7] Y. Liu, J. Gu, N. Goyal, et al., "Multilingual denoising pre-training for neural machine translation," *Transactions of the ACL*, 2020. https://arxiv.org/abs/2001.08210

[8] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: a method for automatic evaluation of machine translation," *Proc. ACL*, 2002. https://aclanthology.org/P02-1040/

[9] M. Popović, "chrF++: words helping character n-grams," *Proc. WMT*, 2017. https://aclanthology.org/W17-4770/

[10] M. Post, "A call for clarity in reporting BLEU scores," *Proc. WMT*, 2018. https://arxiv.org/abs/1804.08771

[11] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *Proc. ICLR*, 2019. https://arxiv.org/abs/1711.05101

[12] T. Kudo, "Subword regularization: improving NMT models with multiple subword candidates," *Proc. ACL*, 2018. https://arxiv.org/abs/1804.10959

[13] Y. Kim and A. M. Rush, "Sequence-level knowledge distillation," *Proc. EMNLP*, 2016. https://arxiv.org/abs/1606.07947

[14] R. Rei, C. Stewart, A. C. Farinha, and A. Lavie, "COMET: A neural framework for MT evaluation," *Proc. EMNLP*, 2020. https://arxiv.org/abs/2009.09025

[15] S. Edunov, M. Ott, M. Auli, and D. Grangier, "Understanding back-translation at scale," *Proc. EMNLP*, 2018. https://arxiv.org/abs/1808.09381

[16] A. Siddhant, A. Bapna, Y. Cao, et al., "Leveraging monolingual data with self-supervision for multilingual neural machine translation," *Proc. ACL*, 2020. https://arxiv.org/abs/2005.04816

[17] NLLB Team, "No language left behind: Scaling human-centered machine translation," *arXiv:2207.04672*, 2022. https://arxiv.org/abs/2207.04672

[18] B. Haddow, R. Bawden, A. V. Miceli Barone, J. Helcl, and A. Birch, "Survey of low-resource machine translation," *Computational Linguistics*, vol. 48, no. 3, pp. 673–732, 2022. https://arxiv.org/abs/2109.00486

[19] F. Alrashidi and H. I. Mathkour, "An empirical study of Transformer-based neural machine translation for English to Arabic," *Information*, vol. 17, no. 2, art. 198, 2026. https://doi.org/10.3390/info17020198

[20] A. Araabi and C. Monz, "Optimizing Transformer for low-resource neural machine translation," *Proc. COLING*, 2020. https://arxiv.org/abs/2011.02266